

INTERNAL TECHNICAL BRIEFING · 13 SEPTEMBER 2026

# Bounding the boundary

---

Running a **local agent fleet** under a token budget, and letting it triage production **without a human in the loop**.

Dan Elliott · a working system, not a proposal — 18 seats, 4 consoles, 993 graded criteria

# Agentic coding is a context-throughput problem long before it is a cost problem

One agent burns context. A **fleet** multiplies it — and the multiplier lands on whatever crosses the host↔worker boundary.

## The naive fleet

Every worker gets the whole repo, the whole ticket history, the whole previous transcript.  $N$  workers,  $N \times$  the context, and no bound anywhere on what one worker can hand back.

## What actually fails first

Not the bill — the *turn*. An oversized brief doesn't run slowly, it runs out: the agent produces a complete module, never wires it to a caller, stops mid-sentence, and reports success.

## So the design question is not "which model"

It is: *what is the maximum number of bytes that may cross this boundary, and what happens at the limit?* Answer that and the model choice becomes a routing decision.

## The claim this deck defends

Bound the boundary in **bytes**, route the bulk of the work to **local** models, and confine paid inference to the two jobs that demonstrably need it.

# cmux-fleet — containerised agents, four consoles, one host



| CONSOLE     | SEATS                      | JOB                               |
|-------------|----------------------------|-----------------------------------|
| operations  | obs-1/2, sre-1/2, tick-1   | read-only cluster work, ticketing |
| development | eng-1/2, tst-1/2           | write code, run suites            |
| review      | col-1, rev-arch/ctx/lang-1 | three-lens review + collation     |
| trriage     | tri-1, obs-t1/t2/t3        | autonomous production sweeps      |

## Triage is the one with no keyboard

The other three consoles are driven by a person at a terminal. Triage has a **host-side actor** that is both its clock and its fan-out performer — it runs unattended, on a cadence, indefinitely.

## Every seat is a container

`uid 10001, --cap-drop ALL , --read-only root, no-new-privileges ,`  
`pids/memory/cpu capped, and an internal Docker network with no route off the host.`

# Bound the boundary, not the model

Four constants decide how much context this system can possibly spend per exchange. They are enforced in code, pinned by tests, and **mirrored inside the container image**.

| CONSTANT                  | DIRECTION      | VALUE   |
|---------------------------|----------------|---------|
| MAX_DISPATCH_TEXT         | brief out      | 32 KiB  |
| MAX_DISPATCH_POLICY_BYTES | drop to worker | 256 KiB |
| MAX_REPLY_ARTIFACT_BYTES  | per file back  | 64 KiB  |
| MAX_REPLY_INLINE_BYTES    | per reply back | 256 KiB |

src/run/dispatch-request.ts:239 · dispatch-policy.ts:176 · relay.ts:269,293

## Outbound refuses. Inbound truncates.

An oversized brief is **rejected whole** with a named code, before anything is written.  
An oversized reply is cut — but never silently.

## The return leg is the same number on purpose

256 KiB out, 256 KiB back. And  $64 \times 4 = 256$  is pinned by an assertion, so the per-file cap cannot drift away from the per-reply cap.

# Enumeration order must not decide which review survives

## The naive truncation

Walk the artifacts in order, inline until the budget runs out, drop the rest. Reviewer three loses its entire review because it happened to be enumerated last — and nothing in the output says so.

## And the cut is named

Each truncated artifact is reported *in the collation brief* with both byte counts, so the collator is told which review was cut and by how much — rather than quietly reading a short one as a brief one.

## What it actually does

**Max-min fair allocation.** Every artifact gets an equal share of the 256 KiB; anything under its share is paid in full, and the surplus is redistributed to the ones over.

## The escape valve is designed, not accidental

A long review is written to a file and declared as a `report`, not pasted inline. The host inlines it under the two caps. The reviewer's own prompt tells it the cap in bytes.

`src/run/relay.ts:326` — `"ENUMERATION ORDER CANNOT DECIDE WHICH REVIEW SURVIVES"`

# Local-first routing, and a ceiling that halts dispatch rather than the world

| PROVIDER     | HOSTED          | SEATS |
|--------------|-----------------|-------|
| omlx         | no — LAN tunnel | 4     |
| gabe         | no — LAN box    | 6     |
| ollama-cloud | yes — billed    | 8     |

**hosted is declared, never inferred**

The local tunnel is HTTPS on a public address — every heuristic would misclassify it as hosted. So the schema **requires** the flag and refuses to guess.

**Paid inference is confined to two jobs**

Code generation and review. Observation, triage, ticketing and SRE work — the high-frequency, unattended traffic — all run on local weights.

**Admission control, not a kill switch**

`tokens_ceiling: 6,000,000` per run. Crossing it halts **dispatch** — artifacts are still harvested, so a budget trip cannot destroy the evidence of what the tokens bought.

**Reservations, not optimism**

Admission projects *spend + all outstanding reservations + this reserve*, because two tasks admitted back-to-back would each individually fit and jointly overshoot.

# Triage — the collator that cannot touch anything

## Grant

`read · grep · find · ls · submit_report · dispatch_request`

**No bash. No write.** `isolation: none` — no code mount at all. No cloud access.  
No egress.

## What it does

Receives the whole declared service list, **divides it** across the three observer seats, briefs each one, and on a second dispatch reconciles the three replies into one document.

## Why it has `dispatch_request` at all

Withdrawing `write` broke the console silently: two sweeps settled `status: success` with the summary "*Dispatched sweep to obs-t1*" — having written no request at all.

A role whose product is a file needs a verb that writes that file. The tool exists because removing the capability produced a **confident, empty success**.

## From its prompt

*"Even matters more than clever. Your observers are not three readings of one thing — they are three workers looking at different services."*

# Observer — the only seat with live cluster access

## Grant

`read · write · bash · grep · find · ls · submit_report`

`cloud_access: true` — but read verbs only, enforced by a wrapper that refuses every mutating verb not on an allowlist.

## Its dominant instruction is a turn budget

*"Around twenty tool calls in, stop investigating and write what you have."*

Because an observer once made **150** `kubectl` calls against exactly the right namespaces — and then ended its turn with an empty outbox.

## It runs only the checks its brief names

Then writes a JSON/Markdown pair. The JSON carries three document-level gates and four per row. Writing only the Markdown clamps the result to `failed`.

## The lesson generalises

An unattended agent's failure mode is not doing the wrong thing. It is doing the right thing until it runs out of turn.

# Ticketing — a live write credential, and a rule about scale

## Grant

`read · write · bash · grep · find · ls · submit_report`

Four secrets, delivered as `*_FILE` paths. `egress_access: true`, scoped to the ticket host. No cloud access.

## From its prompt

*"Ask the server the question. Never download a collection and search it yourself."*

Measured: a run that enumerated **59,616** objects and grepped locally reported that a user with thirty open tickets had **none**.

## Every write is verified by re-fetch

The agent does not trust its own success report. It re-reads the object it claims to have changed.

## Why this is a context lesson, not a correctness one

The agent was not wrong about the API. It was wrong about what fits. Server-side filtering is a *context* control that happens to also be correct.

# Collator — the only one who saw all three

## Grant

`read · grep · find · ls · submit_report · dispatch_request`

`read_only: true , isolation: shared-ro` . No bash, no write. It reads replies through a read-only mount.

## The two rules that define the job

*"Never average them. Your value is in being the only one who saw all three."*

*"You cannot both pre-judge the change and be the one who says the agreement was independent."*

## It does not review

It writes three briefs, is dispatched a second time, and turns three reports into one record.

## Why independence is structural

The collator is forbidden from forming its own verdict first, because a collator with an opinion turns three independent reviews into one review with three witnesses.

# Reviewer — three lenses, three vendors, deliberately

| SEAT       | LENS                         |
|------------|------------------------------|
| rev-arch-1 | architecture & security      |
| rev-ctx-1  | cross-file contract coverage |
| rev-lang-1 | implementation language      |

**Identical config except two fields**

All three are `read_only` , `shared-ro` , same grant. They differ only in `model` and `briefing file`.

**Three vendors is a pinned invariant**

*"Three seats on one model is one reviewer with three transcripts — a shared training blind spot becomes invisible by construction, and independent agreement stops being independent."*

A test fails if the three models ever collapse to one vendor.

**"For every security finding, state the attacker"**

A finding with no reachable attacker is ranked below one with a path. That single instruction removes most of the noise a security lens otherwise produces.

# What a worker can touch

| MOUNT      | MODE                      |
|------------|---------------------------|
| /workspace | git worktree, rw — or :ro |
| /skills    | :ro                       |
| /policy/*  | :ro, root-owned 0555      |
| /replies   | :ro                       |
| /secrets   | :ro                       |
| /outbox    | rw — the only way out     |

## Runtime flags

```
--user 10001:10001
--cap-drop ALL
--security-opt no-new-privileges
--read-only
--tmpfs /tmp:noexec,nosuid
--pids-limit 512 --memory 4g --cpus 2
```

## Zero `-e` flags, asserted

Every environment value arrives via `--env-file` at mode 0600. A test asserts the argv contains **no** inline env flags — so a secret cannot reach the process table.

## Guards run on the finished argv

Not on source literals. Three mount spellings once walked past an earlier check — so the guard now parses the actual command line.

# Three layers, because the first two each have a hole

## 1 · Internal bridge

Docker `--internal` network. The host **refuses to adopt** a pre-existing network of the same name that is not internal, and re-inspects after creation rather than trusting the exit code.

## 2 · Gateway firewall — the hole in layer 1

`--internal` is a FORWARD-chain rule, so the **bridge gateway is still reachable**. Measured: a container pulled an SSH banner off the gateway. Closed with an explicit DROP.

## 3 · Relay + CONNECT proxy

A dual-homed relay carrying only literal host:port pairs, plus a CONNECT proxy that matches on **label boundaries** and default-denies.

### The proxy decides before it resolves

No DNS lookup and no socket is opened before the policy verdict. A denied host is `403` naming the rule — it is never contacted at all.

### It refuses the run rather than degrade

If the firewall rule cannot be *verified*, the run is refused. There is no override flag.

# Files, not environment values — and an honest limit

## Delivery

Secrets are files under `/secrets:ro`. The environment holds only a **pointer**:  
`NAME_FILE=/secrets/NAME`. Writes are verified by read-back — size, mode, type.

## Scoping is an intersection

A fleet-wide allowlist is the ceiling; a role's list is a draw against it. Five distinct refusals, including reserved namespaces and every proxy variable spelling.

## Redaction scrubs the serialised line

Not known fields — the whole JSON — so a new event shape cannot leak silently. It matches leading prefixes too, because a real incident ended in `... | head -c 20`: the model truncated the secret *because it had been told not to echo it*.

## The limit, stated in the source

*"A worker can still cat the file, and anything it cats lands in the same transcript. That is a narrower blast radius, **not a seal**."*

# The verbgate — every cloud verb through one shim

`gcloud`, `kubectl`, `helm`, `gsutil` and `bq` on `PATH` are **not the real binaries** — those are moved to `/usr/local/libexec/<name>.real`. Every invocation, permitted or refused, is classified and appended to a ledger.

| CHECK, IN ORDER               | OUTCOME                                        |
|-------------------------------|------------------------------------------------|
| policy writable               | exit 78 — refuses every verb                   |
| read verb                     | exec the real binary · <code>allow_read</code> |
| on <code>cloud_allow[]</code> | exec · <code>allow_listed</code>               |
| anything else                 | exit 77 · <code>refuse</code>                  |

### What it grants today: nothing

`cloud_allow[]` is designed but **not built**. The policy file is written empty and never rewritten, so every mutating cloud verb is refused — and the config layer *rejects* the key rather than let an operator believe otherwise.

### A control the subject can reconfigure is not a control

Paths are constants, not environment. They used to be overridable, which made the gate self-service: one variable supplied its own policy file, another pointed the ledger at `/dev/null` and discarded the evidence.

### A policy the subject can edit is not a policy

Five surfaces are checked *before* classification, so no path consults a tampered allow-list. A file is tested directly **and through its parent** — a writable `/policy` replaces a 0444 file wholesale.

### Matching is on verb tokens, never the command line

Flags are excluded entirely. A substring match over the whole line is defeated by reordering the flags.

# Five ways it failed open — every one found by writing the attack down

## A positional outvoted the verb

`gcloud compute instances delete real-vm list` classified as a **read**, because a positional was named `list`. It now stops at the first recognised verb, which always precedes its positionals.

## A glob minted credentials

`print-access-token` matched `print-*` and sailed through as a read. `curl` is in the image — so an agent could mint a bearer token and issue any mutating REST call with **zero gate rows and zero refusals**.

## The working directory decided the verdict

An unquoted expansion let an argv of `*` re-expand against the CWD. A worker could drop a file named `describe` beside itself and have a `delete` classified as a read.

## The ledger dropped exactly the evasive rows

`jq 's --args` does not stop option parsing, so any argv carrying a flag made it exit — and the error suppression swallowed the row. The ledger lost precisely the flag-bearing invocations the gate exists to catch.

## A forged task id read back as permitted

Provenance came from environment, which the worker controls. A task id of `T-1`, `"decision":"allow_read` appends a duplicate key and `JSON.parse` takes the last. *"Sanitizing one of three interpolations closed nothing."*

## The ledger is still not authoritative — and says so

It is worker-written, so a host-side collector tails it to a path the worker has no mount to. The guarantee is deliberately the **weaker** of the two available: what is collected survives truncation, and truncation itself is recorded.

# The system refuses in eleven named places

| REFUSAL     | TRIGGER                                              |
|-------------|------------------------------------------------------|
| exit 78     | any policy file writable → refuses <i>every</i> verb |
| exit 77     | mutating verb not on the cloud allowlist             |
| exit 71     | honeypot failed to arm                               |
| exit 73     | credential pointer set but unhonourable              |
| hosted-repo | proprietary remote + hosted model, no consent        |
| gateway     | firewall rule unverifiable                           |

## The stall ladder kills only slot-holders

A worker that is merely queued saturates at *warn* however long it waits — "*killing it would be executing a worker for standing in the line we put it in.*"

## Windows are derived, not configured

A shipped 25-minute kill window against a 900-second deadline meant two live wedges warned and **never reached kill**. The window is now a fraction of the shortest deadline — it can only tighten.

## Identity re-validated at every rung

abort → SIGTERM → SIGKILL, re-checking process identity immediately before each signal, because pids are reused.

# The honeypot — because prevention leaves no record

A blocked escape attempt and a workload that never tried look **identical** in the logs. So the container binds a listener where the Docker socket would be.

## It accepts, records, and destroys

Without reading a byte. *"A detector that answered would be a capability."*

## Three states, not two

`armed`, `0 attempts` is a different fact from `not armed`. Collapsing them would report an unarmed detector as a clean run.

## Its death is fatal to the worker

No config switch, no opt-out. If the detector dies, the seat dies — otherwise the absence of attempts stops meaning anything.

## What this buys

Evidence that the isolation is doing work, rather than an assumption that it is. That distinction is the whole argument for running agents unattended.

# One sweep, end to end

- ▶ **Actor takes a lock** — refuses if another holds it, so two actors cannot sweep one control plane
- ▶ **Mint a sweep id** — re-derived from the run tree, never from the record alone; no id is ever reused
- ▶ **tri-1 partitions** the declared services across three observers — the host validates the partition covers each service *exactly once*
- ▶ **Fan out concurrently** against one deadline, so the sweep costs the largest share rather than the sum
- ▶ **Join, collate, assess** — then announce

## Concurrency is an anti-criterion

Serial dispatch once meant two observers received nothing before the pass ended — and the sweep reported two services it "*could not see*", pointing an operator at a cluster that was healthy.

*"The cost of getting this wrong is not throughput, it is **COVERAGE**."*

## An incomplete partition refuses the whole sweep

A service in no request, or in two, and **nothing is dispatched**. A partial sweep that looks complete is worse than no sweep.

# Three gates that make an unattended verdict trustworthy

## 1 · One row per *declared* service

Not per row the worker wrote. A service the agent omitted still appears — as `indeterminate`. A service it invented cannot appear at all.

## 2 · An unevidenced "healthy" is downgraded

There is **no exported path** that hands a caller the worker's raw `healthy`. The claim must carry evidence or it becomes `indeterminate`.

## 3 · Coverage is counted host-side

From the run tree — never from the worker's own account of what it did.

## And the worker's prose is fenced

The one worker-authored string that reaches a notification is wrapped in banners and line-prefixed, so no agent output can be read as a host statement.

## The shape of the idea

The agent is treated as an **untrusted reporter**. Everything structural is recomputed from facts the host owns.

# There is no dollar figure in this system, and that is deliberate

A local model has no price table. An unpriced run reports `usd: 0` with `priced: false` — not a measured zero.

## The flag is the point

It is what stops a downstream reader averaging **\$0.00** into a spend report as though it were an observation. The only money figure in the repository is this anti-measurement.

## Tokens are the real ceiling

`usd_ceiling` was removed outright rather than left parsed-and-unread.

## The honest caveat

**8 of 18 seats are billed**, and the system says so in prose while attaching no number:  
*"Four panes generating at once ... is not a free console to leave open."*

## What I would claim to a finance partner

Not a saving. A **bound**: a hard cap on bytes per exchange, a token ceiling per run, and 10 of 18 seats that cost electricity rather than API credit.

# Four things I would fix before anyone relied on this

## 1 · The triage console's token ceiling cannot trip

Its accounting reads a state field that **nothing ever writes**, so it sums zeros across four seats. The ceiling is real everywhere else and structurally inert here.

## 2 · `run_timeout` has no reader

Declared, validated, shipped as `2h`, consumed by nothing. Today that is load-bearing — it is why a standing console is possible at all — which makes it a tripwire rather than a bug.

## 3 · Egress is per-fleet, not per-role

The grant buys a *route*, not a destination set. Two workers with egress reach the same hosts. A fleet needing one worker to reach a host another must not **cannot express that**.

## 4 · No retry cap

Re-dispatch is bounded by an epoch fence, not a counter — and that fence does not exist on the attended path.

## Why the gaps are on a slide

Every one of these was found by the system's own criteria file or by a review lens, and each is written down at the point in the source where it would mislead. A guardrail inventory that lists only the working guardrails is marketing.

# Five things I would take to any agent platform

- ▶ **Bound the boundary in bytes, and decide the overflow policy per direction.** Refuse outbound, truncate inbound — fairly, and say what was cut.
- ▶ **Make the agent an untrusted reporter.** Recompute every structural fact host-side: coverage, membership, evidence. Accept prose; never accept counts.
- ▶ **Declare capability, never infer it.** `hosted` is required because every heuristic got it wrong. The same applies to trust, egress and write access.
- ▶ **Instrument the absence.** Prevention leaves no record — so build the detector that turns "nothing happened" into evidence.
- ▶ **Delete config nothing reads.** A parsed-and-unread field advertises a control you do not have, and that is worse than no control.

## The one-line version

A local fleet is affordable because of what it is **forbidden** to send, not because of what it is asked to do.

## DISCUSSION

# Where would you push on this?

---

### **Per-role egress**

Worth the config surface, or is fleet-level the right granularity?

### **Unattended triage**

What would you need to see before it pages someone?

### **Three-vendor review**

Real blind-spot insurance, or expensive theatre?

### **Refusing to price**

Defensible, or a gap dressed as a principle?

Dan Elliott · cmux-fleet